

- Threat Hunt Generation Report: Analyzing Void Dokkaebi's Cython-Compiled InvisibleFerret Malware
 - Executive Summary
 - Threat Actors
 - Void Dokkaebi
 - Campaigns
 - Void Dokkaebi campaign
 - InvisibleFerret Campaign
 - InvisibleFerret Cython Obfuscation Campaign
 - Malware
 - InvisibleFerret (Backdoor)
 - BeaverTail (Loader)
 - Indicators of Compromise
 - Observed Behaviors
 - MITRE ATT&CK Mapping
 - PEAK Hunt Hypotheses
 - Hunt 1
 - Prepare
 - Execute
 - Act
 - Hunt 2
 - Prepare
 - Execute
 - Act

Threat Hunt Generation Report: Analyzing Void Dokkaebi's Cython-Compiled InvisibleFerret Malware

Field	Value
Source	www.trendmicro.com
Published	2026-05-22 00:00 UTC
URL	https://www.trendmicro.com/en_us/research/26/e/analyzing-void-dokkaebi-invisibleferret-malware.html
Classification	Security Incident
Report Generated	2026-06-26 06:30 UTC

Field	Value
Model	cth-qwen:latest
LLM Processing Time	9397.5s

Executive Summary

Void Dokkaebi (Famous Chollima) has updated InvisibleFerret to use Cython-compiled binaries (.pyd on Windows, .so on macOS) for evasion while retaining backdoor, credential theft, and cryptocurrency wallet targeting capabilities. The malware is distributed via obfuscated payloads and leverages fake job interviews to compromise developers with access to CI/CD pipelines and wallet credentials.

Threat Actors

Void Dokkaebi

- **Aliases:** Famous Chollima
- **Motivation:** Targeting software developers with access to cryptocurrency wallet credentials, signing keys, CI/CD pipelines, and production systems
- **Evidence:** Historical use of fake job interviews to lure developers, distribution of Cython-compiled malware

Campaigns

Void Dokkaebi campaign

- **Aliases:** —
- **Description:** Cross-platform malware distribution targeting developers through fake job interviews, leveraging Cython-obfuscated payloads and multi-stage infection chains
- **Evidence:** Infection chain using BeaverTail and InvisibleFerret, URL-based C2 communication patterns

InvisibleFerret Campaign

- **Aliases:** —

- **Description:** Malware distribution using Cython obfuscation and BeaverTail modules for C2 communication and payload execution
- **Evidence:** Downloaded from /clw/{sType} and /clw1/{sType} URLs with dynamic path formatting

InvisibleFerret Cython Obfuscation Campaign

- **Aliases:** —
- **Description:** Campaign utilizing Cython-compiled InvisibleFerret malware with dynamic C2 communication and obfuscation techniques
- **Evidence:** Technical artifacts, TTPs, and code overlaps with BeaverTail and InvisibleFerret

Malware

InvisibleFerret (Backdoor)

- **Aliases:** —
- **Description:** Cython-compiled malware distributed as .pyd (Windows) and .so (macOS) files, retaining core capabilities like credential theft, keylogging, and cryptocurrency wallet targeting

BeaverTail (Loader)

- **Aliases:** —
- **Description:** Multi-stage downloader and stealer expanded into a broader malware family with backdoor, browser-stealing, and trojanized wallet installation modules

Indicators of Compromise

IOC Value	Type	Context
/client/{sType}	URL	Original download path for InvisibleFerret (main) in previous samples

IOC Value	Type	Context
/clw/{sType}	URL	Download path for InvisibleFerret in current campaign; Windows-based C2 communication path for InvisibleFerret; Dynamic
/clw1/{sType}	URL	Download path for non-Windows InvisibleFerret payloads; Non-Windows-based C2 communication path for InvisibleFerret; Dyn
main_{sType}.py	Filename	Saved filename for previously observed InvisibleFerret (main) payloads
.mod	Filename	Execution file created by BeaverTail to run Cython-compiled payloads; Python script created and executed by BeaverTail t

IOC Value	Type	Context
.pyd	Filename	Cython-compiled Windows payload distribution format
.so	Filename	Cython-compiled macOS payload distribution format
mod.pyd	Filename	Cython binary downloaded by BeaverTail (gjs) for Windows execution; Cython-compiled binary file for Windows; Cython bina
mod.so	Filename	Cython binary downloaded by BeaverTail (gjs) for macOS execution; Cython-compiled binary file for macOS; Cython binary f
PyInIt_mod	Filename	Export table entry in Windows InvisibleFerret binaries

IOC Value	Type	Context
/Users/administrator/Pictures/Work/py_module_work/build/temp.macosx-10.13-universal2-cpython-312/mod.o	Filename	Object file path in macOS InvisibleFerret binaries
Zlib-compressed string constants	Filename	Embedded in Cython-obfuscated InvisibleFerret binaries
mc.so	Filename	Module that deletes execution script after use; Malware module targeting cryptocurrency wallets
pad0	Filename	Python script for executing next-stage payloads
brw0	Filename	Python script for executing next-stage payloads
Chrome (downgraded version)	Filename	Downgraded Chrome used to bypass Manifest V3 restrictions

Observed Behaviors

1. Python script executed to run Cython-compiled InvisibleFerret payloads

- **Evidence:** Cython-generated binaries require a Python script or interpreter to load them
- **Artifacts:** Python script, .mod, .pyd, .so

- **Context:** Infection chain relies on Python execution scripts to bypass binary-only detection

2. BeaverTail obfuscates strings using array shuffling and hexadecimal index lookup

- **Evidence:** Large array of Base64 fragments shuffled using IIFE and retrieved via hexadecimal index lookup
- **Artifacts:** Base64 fragments, hexadecimal index, IIFE
- **Context:** Obfuscation technique to evade simple string detection

3. BeaverTail XOR-encrypts sensitive strings with 4-byte key

- **Evidence:** Most sensitive strings (file paths, commands) are XOR-encrypted using a 4-byte key
- **Artifacts:** XOR encryption, 4-byte key
- **Context:** Protection for critical execution commands and file paths

4. BeaverTail splits and swaps C2 IP addresses before Base64 decoding

- **Evidence:** C&C IP addresses are split into two halves, swapped, and encoded with Base64
- **Artifacts:** Base64 encoding, split IP addresses, swapped halves
- **Context:** Evasion technique to obscure C2 communication patterns

5. InvisibleFerret downloaded via URL-based C2 with dynamic path formatting

- **Evidence:** InvisibleFerret is downloaded from /clw/{sType} (Windows) or /clw1/{sType} (non-Windows)
- **Artifacts:** URL, /clw/{sType}, /clw1/{sType}
- **Context:** C2 communication paths use dynamic identifiers to avoid static detection

6. BeaverTail (gjs) downloads Cython binary (mod.pyd/mod.so) from C2 server

- **Evidence:** BeaverTail (gjs) downloads the Cython binary (mod.pyd or mod.so) from the C&C server
- **Artifacts:** mod.pyd, mod.so, C2 server
- **Context:** Execution flow initiated by _rum() function for cross-platform infection

7. Python script (.mod) created and executed to load Cython binaries

- **Evidence:** BeaverTail creates and executes a .mod script to import mod.pyd/mod.so as Python extension modules
- **Artifacts:** .mod, Python interpreter, mod.pyd, mod.so
- **Context:** Required due to Cython binaries relying on CPython runtime

8. Command-line arguments passed to Cython module include sType, encoded IPs, and port number

- **Evidence:** The .mod script passes sType, encoded IP addresses, and a port number as command-line arguments
- **Artifacts:** sType, encoded IP addresses, port number
- **Context:** Used for deobfuscating Python payload and establishing C2 communication

9. Zlib-decompressed string table contains XOR-encoded IP addresses and module identifiers

- **Evidence:** String table decompressed with Zlib reveals [XOR Encoded IP address] ? [Module Name].pyx format
- **Artifacts:** Zlib, XOR Encoded IP address, Module Name.pyx
- **Context:** Used to reconstruct original Python payload from compiled binaries

10. dcp function reconstructs IP address from 16-character XOR-encoded string

- **Evidence:** The dcp function uses an XOR-based routine to decode 16-character strings into IP addresses
- **Artifacts:** XOR encoding, 16-character string
- **Context:** Allows dynamic determination of C2 destination at runtime

11. Execution Python scripts (e.g., .mod) pass sType, encoded IPs, and port numbers to Cython binaries

- **Evidence:** The .mod script passes sType, encoded IP addresses, and a port number as command-line arguments
- **Artifacts:** sType, encoded IP addresses, port number
- **Context:** Used for deobfuscating Python payload and establishing C2 communication

12. mc.so module deletes execution Python script after use

- **Evidence:** Some variants delete the execution Python script after use
- **Artifacts:** mc.so
- **Context:** Conceals connection destinations and reduces forensic evidence

13. InvisibleFerret targets cryptocurrency wallets including MetaMask, Coinbase, and Phantom

- **Evidence:** The original InvisibleFerret (mc) targeted only MetaMask, but current mc.so now also targets Coinbase and Phantom
- **Artifacts:** Coinbase, MetaMask, Phantom
- **Context:** Attackers modify Chrome versions to bypass Manifest V3 restrictions

14. Chrome downgraded on macOS to support Manifest V2 for Chrome extensions

- **Evidence:** Attackers downgrade the version of Chrome on macOS to bypass Google's enforced transition to Manifest V3
- **Artifacts:** Chrome (downgraded version)

- **Context:** Enables trojanized cryptocurrency wallet components as Chrome extensions

15. Legacy AnyDesk execution environment contains Base64-decoded IP address code

- **Evidence:** The deobfuscated InvisibleFerret (any) code still contains legacy code decoding 20-character Base64-encoded string
- **Artifacts:** Base64 encoding, 20-character string
- **Context:** Legacy code fails due to commented-out Base64 string

MITRE ATT&CK Mapping

Tactic	Technique ID	Technique Name	Observed Behavior
Execution	T1059.006	Command and Scripting Interpreter: Python	Python script executed to run Cython-compiled InvisibleFerret payloads
Stealth	T1027.008	Obfuscated Files or Information: Stripped Payloads	BeaverTail obfuscates strings using array shuffling and hexadecimal in
Defense Impairment	T1600.001	Weaken Encryption: Reduce Key Space	BeaverTail XOR-encrypts sensitive strings with 4-byte key
Defense Impairment	T1599.001	Network Boundary Bridging: Network Address Translation Traversal	BeaverTail splits and swaps C2 IP addresses before Base64 decoding
Stealth	T1027.012	Obfuscated Files or Information: LNK Icon Smuggling	InvisibleFerret downloaded via URL-based C2 with dynamic path format
Command and Control	T1105	Ingress Tool Transfer	BeaverTail (gjs) downloads Cython binary (mod.pyd/mod.so) from C2 serv
Execution	T1059.006	Command and Scripting Interpreter: Python	Python script (.mod) created and executed to load Cython binaries
Command and Control	T1568.003	Dynamic Resolution: DNS Calculation	Command-line arguments passed to Cython module include sType, encoded
Stealth	T1027.008	Obfuscated Files or Information: Stripped Payloads	Zlib-decompressed string table contains XOR-encoded IP addresses and m

Tactic	Technique ID	Technique Name	Observed Behavior
Command and Control	T1568.003	Dynamic Resolution: DNS Calculation	dcp function reconstructs IP address from 16-character XOR-encoded str
Command and Control	T1568.003	Dynamic Resolution: DNS Calculation	Execution Python scripts (e.g., .mod) pass sType, encoded IPs, and por
Stealth	T1070.009	Indicator Removal: Clear Persistence	mc.so module deletes execution Python script after use
Impact	T1496.001	Resource Hijacking: Compute Hijacking	InvisibleFerret targets cryptocurrency wallets including MetaMask, Coi
Defense Impairment	T1689	Downgrade Attack	Chrome downgraded on macOS to support Manifest V2 for Chrome extension
Command and Control	T1132.001	Data Encoding: Standard Encoding	Legacy AnyDesk execution environment contains Base64-decoded IP address

PEAK Hunt Hypotheses

Hunt 1

Prepare

- **Hypothesis:** Adversaries may be using Python scripts to execute Cython-compiled payloads for C2 communication and persistence.
- **Behavior Basis:** Python script executed to run Cython-compiled InvisibleFerret payloads, with artifacts like .mod, .pyd, .so files and dynamic URL-based C2 paths.
- **Objective:** Validate if Python scripts are being used to load and execute Cython binaries for C2 operations.
- **Required Data Sources:** Process creation logs, PowerShell execution logs, Network connection telemetry

Execute

Gather Data

- Collect process creation events for Python scripts (.py) and Cython binaries (.mod, .pyd, .so)
- Capture network connections from Python scripts to URLs matching /clw/{sType} or /clw1/{sType}
- Extract command-line arguments passed to Cython modules for sType, encoded IPs, and ports

Analysis Steps

- Correlate Python script execution with network connections to C2 domains
- Check if Cython binaries (.mod, .pyd, .so) are loaded via Python interpreter
- Validate if command-line arguments contain XOR-encoded IPs or dynamic C2 paths

Supporting Evidence

- Python script execution followed by Cython binary loading
- Network traffic to C2 URLs with /clw/{sType} or /clw1/{sType} paths
- Presence of XOR-encoded IPs in command-line arguments

Act

- **Documentation Requirements:** Record confirmed Python script execution, Cython binary loading, and C2 communication patterns.

Findings to Preserve

- Python scripts used to load Cython binaries
- C2 URLs with dynamic path formatting
- XOR-encoded IP arguments in command lines

Future Hunt Recommendations

- Monitor for Python scripts loading non-standard libraries
- Track anomalies in command-line arguments for encoded IPs
- Investigate process creation events for suspicious .mod/.pyd/.so files

Hunt 2

Prepare

- **Hypothesis:** Adversaries may be using obfuscated strings and split IP addresses to evade detection during C2 operations.
- **Behavior Basis:** BeaverTail obfuscates strings via array shuffling, hexadecimal indexing, and XOR encryption, with split/swap IP addresses encoded in Base64.
- **Objective:** Identify obfuscated strings and split IP addresses used for C2 communication.

- **Required Data Sources:** Process creation logs, Memory artifacts, String decryption telemetry

Execute

Gather Data

- Collect memory dumps of processes executing BeaverTail (gjs) or Cython modules
- Extract Base64-encoded strings and hexadecimal index patterns from process memory
- Search for XOR-encrypted strings with 4-byte keys in script artifacts

Analysis Steps

- Decode Base64 fragments using hexadecimal index lookup to reconstruct strings
- Apply XOR decryption with 4-byte keys to sensitive strings (file paths, IPs)
- Check if split IP addresses are reassembled into valid C2 destinations

Supporting Evidence

- Base64 fragments shuffled with hexadecimal index lookup
- XOR-encrypted strings decrypted to reveal C2 IPs or file paths
- Split IP addresses reassembled into valid network destinations

Act

- **Documentation Requirements:** Document obfuscation techniques, decrypted strings, and reassembled C2 IPs.

Findings to Preserve

- Obfuscated strings decoded via array shuffling and hex indexes
- XOR-encrypted strings decrypted with 4-byte keys
- Split IP addresses reassembled into C2 server destinations

Future Hunt Recommendations

- Monitor for Base64-encoded strings with irregular hex index patterns
- Track XOR decryption attempts with non-standard key lengths
- Investigate memory artifacts for split IP address patterns